Informe de Pruebas
Clúster de instrumentos simulado del Ferrari F40 (1987)

Software simulado, sin ECU ni sensores reales. Los rangos y umbrales son parámetros didácticos, no calibraciones oficiales del F40.

1. Portada e identificación

Campo	Valor
Institución	Universidad Autónoma de Guadalajara (UAG)
Materia	Sistemas Operativos
Profesor	José de Jesús Velázquez Martínez
Equipo	Equipo 3
Integrantes	César Saúl Vázquez Preciado (5079359) Ian Josué Cota Flores (5060667)
Fecha de entrega	18 de octubre de 2026
Fecha de ejecución de las pruebas	2026-11-10
Versión del software probado	commit af7290f
Colaboración	Claude Code (Anthropic), asistente de inteligencia artificial; ver sección 14
Documento generado por	informe/generar_informe.py a partir de las evidencias en docs/

Contenido

- 1. Portada e identificación
- 2. Vehículo y descripción del clúster
- 3. Los diez elementos y las reglas
- 4. Alcance y exclusiones
- 5. Ambiente y datos de prueba
- 6. Niveles, tipos y técnicas aplicadas
- 7. Herramientas
- 8. Riesgos
- 9. Casos de prueba
- 10. Matriz de trazabilidad
- 11. Registro de defectos
- 12. Resumen cuantitativo y conclusión de calidad
- 13. Referencias
- 14. Anexo: uso de inteligencia artificial

2. Vehículo y descripción del clúster

El sistema bajo prueba simula el clúster de instrumentos de un Ferrari F40 (1987). No se conecta a una ECU ni a sensores: los valores se introducen con controles en pantalla (5 deslizadores, 4 interruptores y un selector de direccional) y el clúster los muestra en 5 medidores de aguja y 5 testigos luminosos. Los rangos y umbrales son parámetros didácticos definidos por la especificación del curso.

La interfaz tiene una estética 8-bit inspirada en la distribución del tablero del F40: velocímetro y tacómetro grandes al centro, medidores chicos a los lados y los dos testigos críticos arriba, entre las esferas. Agujas, esferas y escalas se dibujan con bloques de pixeles; los testigos 6, 7 y 8 van dentro de las esferas. Se abre en Windows, Linux y macOS con Python 3.10 o superior (ver README.md).

Arquitectura del software:

Módulo	Responsabilidad
logic/estado.py	Estado del clúster y reglas R01–R08; valida las entradas; sin dependencias de interfaz.
logic/reloj.py	Reloj inyectable: real (time.monotonic) o simulado para pruebas deterministas.
ui/presentacion.py	Traduce el estado a lo que se dibuja de cada uno de los 10 elementos.
ui/tablero.py	Interfaz Tkinter: dibuja medidores y testigos y envía los controles al estado.
main.py	Punto de entrada de la aplicación.
aceptacion/escenarios.py	Carga y ejecuta los escenarios de aceptación desde un archivo CSV.
sistema/lector.py	Hilo lector de datos/vehiculo.conf: valida y publica el último estado válido.
sistema/bitacora.py, sistema/monitor.py	Bitácoras eventos.log y errores.log con rotación; eventos de seguridad.
sistema/config.py	Configuración externa: rutas, intervalos y rangos (config/cluster.ini).

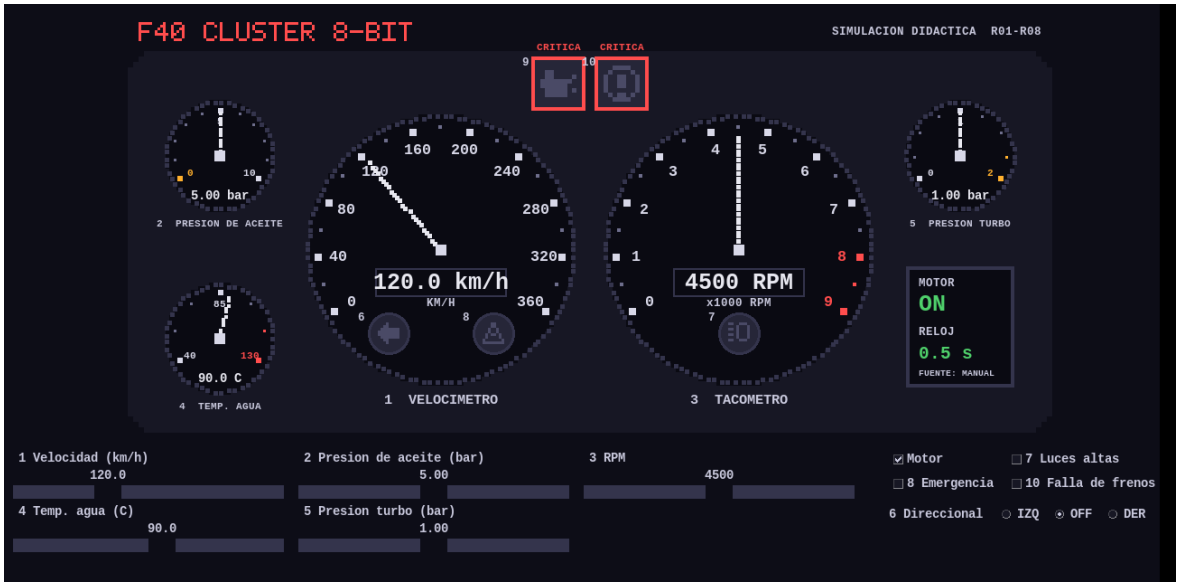


Figura 1. Vista general del clúster con el motor encendido (captura 02).

3. Los diez elementos y las reglas

Categorías: 1–5 lecturas continuas; 6–8 testigos informativos; 9–10 testigos de seguridad. El testigo 9 es distinto del medidor continuo 2.

N.º	Elemento	Tipo	Rango	Ciclo	Condición	Comportamiento	Categoría
1	Velocímetro	Decimal	0–360 km/h	100 ms	Encendido = 1	Aguja y cifra	Lectura continua
2	Presión de aceite	Decimal	0–10 bar	100 ms	Motor activo	Aguja; ámbar si < 1 bar	Lectura continua
3	Tacómetro	Entero	0–9000 RPM	100 ms	Motor activo	Rojo si ≥ 8000 RPM	Lectura continua
4	Temperatura de agua	Decimal	40–130 °C	500 ms	Encendido = 1	Rojo si > 110 °C	Lectura continua
5	Presión de turbo	Decimal	0–2 bar	100 ms	Motor activo	Contorno ámbar si > 1.5 bar	Lectura continua
6	Direccionales	OFF / IZQ / DER	—	500 ms	—	Verde intermitente	Testigo informativo
7	Luces altas	Booleano	—	—	—	Azul fijo	Testigo informativo
8	Intermitentes (emergencia)	Booleano	—	500 ms	—	Ámbar intermitente	Testigo informativo
9	Baja presión de aceite	Booleano	—	—	Motor activo (R02)	Rojo fijo	Testigo de seguridad (CRÍTICA)
10	Falla de frenos	Booleano	—	300 ms	—	Rojo parpadeante	Testigo de seguridad (CRÍTICA)

Reglas de funcionamiento

Regla	Descripción
R01	encendido = 1: actualizar velocidad, RPM, temperatura, presión de aceite y turbo según su ciclo (100/500 ms). encendido = 0: velocidad y RPM a 0; las demás lecturas en un estado definido.
R02	motor_activo = 1 y p_aceite < 1.0 bar durante ≥ 1 s: encender el testigo 9 en rojo. p_aceite ≥ 1.2 bar durante ≥ 1 s: apagarlo (histéresis).
R03	rpm ≥ 8000: indicador 3 en rojo. rpm < 7800: color normal.
R04	temp_agua > 110 °C: indicador 4 en rojo. ≤ 105 °C: normal.
R05	p_turbo > 1.5 bar: contorno del indicador 5 en ámbar. ≤ 1.4 bar: sin advertencia.
R06	Direccional IZQ o DER: la lámpara 6 alterna cada 500 ms. OFF: apagada.
R07	luces_altas = 1: testigo 7 azul fijo. emergencia = 1: el testigo 8 parpadea cada 500 ms y tiene precedencia visual sobre el testigo de giro.
R08	falla_frenos = 1: testigo 10 rojo parpadeante cada 300 ms hasta falla_frenos = 0.

4. Alcance y exclusiones

Dentro del alcance

- Reglas R01–R08 y validación de entradas en logic/estado.py; reloj en logic/reloj.py.
- Presentación de los 10 elementos (ui/presentacion.py) e interfaz Tkinter (ui/tablero.py, main.py), probadas con Tkinter real sobre una pantalla virtual (Xvfb).
- Prueba de aceptación dirigida por datos (aceptacion/escenarios.py) y manejo de su archivo de escenarios.
- Evidencia visual: capturas reales de la ventana en estados representativos.

Fuera del alcance

- Hardware, ECU, sensores y calibraciones reales del vehículo.
- Rendimiento, carga y precisión temporal del reloj real.
- Otras plataformas: solo se probó en la máquina virtual descrita en la sección 5.
- Ejecución interactiva prolongada de main.py sin la opción --salir-tras-ms (se recomienda una prueba manual).
- Demostrar conformidad con ISO 26262; la norma solo se usa como contexto.

Decisiones de interpretación

Puntos donde la especificación admite más de una lectura. Fueron propuestos durante el análisis y aprobados por el equipo; cada uno tiene casos que lo verifican.

ID	Decisión
D1	R08 no condiciona el testigo 10 al motor: parpadea aunque el motor esté apagado. El defecto DEF-01, que suponía lo contrario, se rechazó.
D2	Los testigos informativos 6, 7 y 8 solo se muestran con el motor encendido; es el estado definido que R01 deja abierto.
D3	La especificación distingue Encendido (elementos 1 y 4) y Motor activo (2, 3 y 5); la interfaz tiene un solo interruptor Motor que representa ambas señales.
D4	La prueba de aceptación lee sus escenarios de un archivo CSV; así se prueba también el manejo de archivos (inexistente, vacío, columnas faltantes, filas inválidas).
D5	Este informe presenta 25 casos consolidados (CT-01 a CT-25); cada uno se respalda en pruebas automatizadas (CP-xx) y de aceptación (CA-xx). La matriz completa está en docs/.
D6	La portada usa la fecha de entrega de la especificación; las evidencias conservan su fecha real de ejecución.
D7	Precedencia de R07: mientras la emergencia está activa, el testigo 6 no se ilumina y solo parpadea el 8.
R02*	"Durante ≥ 1 s" se interpreta como tiempo continuo: entre 1.0 y 1.2 bar se reinician ambos temporizadores, por lo que una presión que oscila entre 0.9 y 1.1 bar no enciende el testigo 9.

5. Ambiente y datos de prueba

Ambiente de ejecución

Componente	Versión
Sistema operativo	Ubuntu 22.04.5 LTS (kernel 5.15.0-190-generic)

Componente	Versión
Python	3.10.12
Tk	8.6
pytest	9.2.0
coverage.py	7.16.2
Pillow	12.3.0
python-docx	1.2.0
Pantalla	Xvfb (pantalla virtual); capturas a 1280x1024, 24 bits

Preparación

```
sudo apt-get install -y python3-tk python3-venv xvfb
sudo apt-get install -y --no-install-recommends libreoffice-writer-nogui fonts-liberation
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
Abrir la aplicación: bash abrir_cluster.sh (Linux/macOS) o doble clic en abrir_cluster.bat
(Windows)
```

Archivos y datos de prueba

Archivo	Contenido
tests/conftest.py	Fixtures: reloj simulado, estado inicial y estado con motor encendido.
tests/test_*.py	Pruebas automatizadas CP-01 a CP-90 y CA-01 a CA-10.
tests/datos/escenarios_aceptacion.csv	Escenarios de aceptación: control, valor, tiempo, elemento y resultado esperado de cada paso.
tools/capturas.py	Generación de capturas en estados representativos.
tools/trazabilidad.py	Matriz de trazabilidad desde los docstrings y junit.xml.

Ejecución

```
xvfb-run -a python -m coverage run --branch -m pytest -v --junitxml=docs/evidencias/junit.xml
python -m coverage report -m
xvfb-run -a python -m aceptacion.escenarios tests/datos/escenarios_aceptacion.csv
xvfb-run -a -s "-screen 0 1280x1024x24" python tools/capturas.py
python tools/trazabilidad.py
python informe/generar_informe.py
```

Criterios de evaluación

Estado	Criterio
Aprobada	Todas las ejecuciones que respaldan el caso pasaron y, si incluye escenarios de aceptación, todos sus pasos coinciden con lo esperado en pantalla.
Fallida	Al menos una ejecución que respalda el caso falló.
Bloqueada	No hay ejecución registrada de alguna prueba de respaldo, o se omitió (por ejemplo, sin servidor gráfico).

6. Niveles, tipos y técnicas aplicadas

Nivel	Qué se prueba
Unitario	logic/estado.py y logic/reloj.py de forma aislada, con reloj simulado.
Integración	ui/presentacion.py y ui/tablero.py con Tkinter real; main.py como prueba de humo.
Sistema / aceptación	Escenarios completos sobre la ventana real, observando lo dibujado.

Técnica	Ejecuciones	Aprobadas	Fallidas	Omitidas
Aceptacion (dirigida por datos)	27	27	0	0
Caja blanca (cobertura)	13	13	0	0
Entradas invalidas	15	15	0	0
Valores limite	16	16	0	0
Particion de equivalencia	10	10	0	0
Presentacion (caja negra)	7	7	0	0
Basada en requisitos	3	3	0	0
Sistema (archivos, bitacoras, concurrencia)	37	37	0	0
Integracion UI (Tkinter)	6	6	0	0
Transicion de estados	11	11	0	0

Tipos de prueba: funcional (caja negra), estructural (caja blanca con cobertura de ramas), regresión (toda la suite se ejecuta tras cada cambio), manejo de errores y archivos, y aceptación dirigida por datos.

Marco normativo

El proceso y este documento se inspiran en la serie ISO/IEC/IEEE 29119; ISO 26262-6 se usa solo como contexto automotriz. No se declara conformidad con ninguna de ellas.

Norma	Tema	Cómo se aplicó en este proyecto
ISO/IEC/IEEE 29119-1:2022	Conceptos generales de prueba	Vocabulario del informe: caso de prueba, oráculo, cobertura, defecto, niveles y tipos de prueba.
ISO/IEC/IEEE 29119-2:2021	Procesos de prueba	El trabajo siguió las etapas de planificación (auditoría y alcance), diseño e implementación (casos CP/CA), ejecución con registro de resultados, reporte de incidentes (DEF-01) y cierre (este informe).
ISO/IEC/IEEE 29119-3:2021	Documentación de prueba	Estructura de este documento: alcance, ambiente y datos, especificación de casos con resultado esperado y obtenido, registro de defectos, trazabilidad y resumen de completitud.
ISO/IEC/IEEE 29119-4:2021	Técnicas de prueba	Técnicas basadas en especificación (partición de equivalencia, valores límite, transición de estados) y basadas en estructura (cobertura de sentencias y ramas).
ISO 26262-6:2018	Seguridad funcional, software automotriz	Solo como contexto: distinción entre testigos críticos (9 y 10) e informativos y verificación por requisitos. No se pide ni se demuestra conformidad.

7. Herramientas

Herramienta	Versión	Uso	Fuente oficial
Python	3.10.12	Lenguaje y ejecución	https://docs.python.org/3/library/tkinter.html
Tkinter (Tk)	8.6	Interfaz gráfica	https://docs.python.org/3/library/tkinter.html
pytest	9.2.0	Ejecución de pruebas automatizadas	https://docs.pytest.org/en/stable/
coverage.py	7.16.2	Cobertura de líneas y ramas	https://coverage.readthedocs.io/
Pillow	12.3.0	Capturas de pantalla	https://pillow.readthedocs.io/
python-docx	1.2.0	Generación de este informe	https://python-docx.readthedocs.io/
Xvfb	paquete xvfb de Ubuntu	Pantalla virtual para la interfaz	https://manpages.ubuntu.com/manpages/jammy/man1/Xvfb.1.html
LibreOffice	modo headless (soffice)	Conversión del informe a PDF	https://www.libreoffice.org/

8. Riesgos

Riesgo	Impacto	Mitigación
Interpretación de la especificación (D1–D7)	Alto	Decisiones documentadas y aprobadas por el equipo; cada una tiene casos que la verifican.
Los valores esperados se generaron con la misma IA que escribió el código	Alto	Valores derivados directamente de los umbrales de R01–R08; CP-90 demuestra que el oráculo detecta diferencias; revisión del equipo.
La interfaz se probó en una pantalla virtual (Xvfb), no en un monitor real	Medio	Capturas reales de la ventana; se recomienda una ejecución manual con python main.py.
El tiempo se simula en las pruebas; no se midió la precisión del reloj real	Medio	CP-60 y CP-78 prueban el reloj real; el rendimiento queda fuera del alcance.
Umbrales con decimales (110.1 °C, 1.51 bar, 0.99 bar)	Bajo	Controles redondeados a su resolución y casos en ambos lados de cada umbral.
Un solo interruptor para Encendido y Motor activo (D3)	Bajo	Restricción documentada en el alcance.
Cobertura alta no garantiza ausencia de defectos	Medio	Se combinan técnicas de caja negra, aceptación y revisión de la especificación.

9. Casos de prueba

Los 25 casos CT-01 a CT-25 consolidan las pruebas ejecutadas. Cada uno indica las pruebas automatizadas (CP-xx) y de aceptación (CA-xx) que lo respaldan; el resultado obtenido y el estado se calculan de docs/evidencias/junit.xml y aceptacion.txt.

CT-01 — Estado inicial seguro

Identificador	CT-01
Requisitos	R01
Elementos	1–10

Técnica	Funcional, caja negra (aceptación)
Precondiciones	Aplicación recién iniciada; ningún control manipulado.
Datos de prueba	Ninguno (valores por defecto).
Pasos	1. Iniciar la aplicación (python main.py). 2. Sin tocar los controles, observar los 10 elementos y los controles.
Resultado esperado	Agujas 1–5 en su mínimo (0 km/h, 0 bar, 0 RPM, 40 °C, 0 bar) en color normal; testigos 6–10 apagados; interruptor Motor apagado y direccional en OFF.
Resultado obtenido	3 ejecuciones automatizadas (CP-68, CP-74, CA-01): 3 aprobadas, 0 fallidas, 0 omitidas. CA-01: APROBADO (6/6 pasos verificados en pantalla).
Estado	**Aprobada**
Evidencia	Captura 01; docs/evidencias/junit.xml

CT-02 — Encendido: actualización de lecturas según su ciclo

Identificador	CT-02
Requisitos	R01
Elementos	1, 3
Técnica	Partición de equivalencia, transición de estados
Precondiciones	Motor apagado.
Datos de prueba	velocidad = 120.5 km/h; RPM = 4500; motor = 1
Pasos	1. Con el motor apagado fijar 120.5 km/h y comprobar que el velocímetro sigue en 0. 2. Encender el motor. 3. Observar el velocímetro a los 50 ms. 4. Observarlo a los 100 ms.
Resultado esperado	Con el motor apagado no hay actualización; tras encenderlo, el valor aparece al cumplirse el ciclo de 100 ms y no antes.
Resultado obtenido	4 ejecuciones automatizadas (CP-01, CP-02, CP-35, CA-01): 4 aprobadas, 0 fallidas, 0 omitidas. CA-01: APROBADO (6/6 pasos verificados en pantalla).
Estado	**Aprobada**
Evidencia	Captura 02; docs/evidencias/junit.xml

CT-03 — Apagado: velocidad y RPM a 0 y estado definido

Identificador	CT-03
Requisitos	R01, R02
Elementos	1–5, 9
Técnica	Transición de estados
Precondiciones	Motor encendido con lecturas distintas de cero.
Datos de prueba	velocidad 200 km/h, 5000 RPM, aceite 5 bar, turbo 1 bar, temperatura 90 °C
Pasos	1. Fijar los valores indicados y esperar 500 ms. 2. Apagar el motor. 3. Observar las lecturas y el testigo 9.
Resultado esperado	Velocidad y RPM en 0; aceite y turbo en 0; temperatura en 40 °C; testigo 9 apagado y su temporizador reiniciado.
Resultado obtenido	3 ejecuciones automatizadas (CP-36, CP-37, CA-03): 3 aprobadas, 0 fallidas, 0 omitidas. CA-03: APROBADO (8/8 pasos verificados en pantalla).
Estado	**Aprobada**
Evidencia	docs/evidencias/aceptacion.txt (CA-03); docs/evidencias/junit.xml

CT-04 — Velocímetro: valores límite del rango

Identificador	CT-04
Requisitos	R01
Elementos	1
Técnica	Valores límite
Precondiciones	Motor encendido.
Datos de prueba	0 y 360 km/h (válidos); −1 y 361 km/h (inválidos)
Pasos	1. Fijar 0 km/h y esperar 100 ms. 2. Fijar 360 km/h y esperar 100 ms. 3. Intentar fijar −1 y 361 km/h.
Resultado esperado	0 y 360 km/h se muestran; −1 y 361 km/h se rechazan con ValueError sin cambiar el valor.
Resultado obtenido	5 ejecuciones automatizadas (CP-10, CP-11, CP-12, CP-38, CP-39): 5 aprobadas, 0 fallidas, 0 omitidas.
Estado	**Aprobada**
Evidencia	docs/evidencias/pytest.txt; docs/evidencias/junit.xml

CT-05 — Presión de aceite (medidor 2): ámbar inmediato bajo 1 bar

Identificador	CT-05
Requisitos	R01
Elementos	2
Técnica	Valores límite, transición de estados
Precondiciones	Motor encendido.
Datos de prueba	5, 0.99 y 1.0 bar
Pasos	1. Fijar 5 bar y esperar 100 ms. 2. Bajar a 0.99 bar y esperar 100 ms. 3. Subir a 1.0 bar y esperar 100 ms. 4. Apagar el motor.
Resultado esperado	Normal en 5 bar; ámbar en 0.99 bar sin esperar 1 s; normal en 1.0 bar; normal con el motor apagado.
Resultado obtenido	5 ejecuciones automatizadas (CP-03, CP-54, CP-55, CP-56, CA-02): 5 aprobadas, 0 fallidas, 0 omitidas. CA-02: APROBADO (6/6 pasos verificados en pantalla).
Estado	**Aprobada**
Evidencia	Captura 03; docs/evidencias/junit.xml

CT-06 — Tacómetro: rojo a partir de 8000 RPM

Identificador	CT-06
Requisitos	R03
Elementos	3
Técnica	Valores límite
Precondiciones	Motor encendido.
Datos de prueba	7799 y 8000 RPM
Pasos	1. Fijar 7799 RPM y esperar 100 ms. 2. Fijar 8000 RPM y esperar 100 ms.
Resultado esperado	7799 RPM en color normal; 8000 RPM con el indicador 3 en rojo.
Resultado obtenido	3 ejecuciones automatizadas (CP-13, CP-16, CA-03): 3 aprobadas, 0 fallidas, 0 omitidas. CA-03: APROBADO (8/8 pasos verificados en pantalla).
Estado	**Aprobada**
Evidencia	Captura 03; docs/evidencias/junit.xml

CT-07 — Tacómetro: histéresis entre 7800 y 7999 RPM

Identificador	CT-07

Requisitos	R03
Elementos	3
Técnica	Transición de estados (histéresis)
Precondiciones	Motor encendido.
Datos de prueba	7800, 8000, 7999, 7900 y 7799 RPM
Pasos	1. Partiendo de normal, fijar 7800 RPM. 2. Subir a 8000 RPM. 3. Bajar a 7999 y luego a 7900 RPM. 4. Bajar a 7799 RPM.
Resultado esperado	En 7800 conserva normal; en 8000 pasa a rojo; en 7999 y 7900 conserva rojo; en 7799 vuelve a normal.
Resultado obtenido	4 ejecuciones automatizadas (CP-14, CP-15, CP-27, CA-03): 4 aprobadas, 0 fallidas, 0 omitidas. CA-03: APROBADO (8/8 pasos verificados en pantalla).
Estado	**Aprobada**
Evidencia	docs/evidencias/aceptacion.txt (CA-03); docs/evidencias/junit.xml

CT-08 — Temperatura de agua: rojo sobre 110 °C y ciclo de 500 ms

Identificador	CT-08
Requisitos	R04
Elementos	4
Técnica	Valores límite
Precondiciones	Motor encendido.
Datos de prueba	90, 110, 110.1 y 120 °C
Pasos	1. Fijar 90 °C y esperar 500 ms. 2. Fijar 110 °C y esperar 500 ms. 3. Fijar 110.1 °C y esperar 500 ms. 4. Desde normal, fijar 120 °C y observar a los 250 y a los 500 ms.
Resultado esperado	90 y 110 °C en normal; 110.1 °C en rojo; el cambio a 120 °C se refleja solo al cumplirse 500 ms.
Resultado obtenido	4 ejecuciones automatizadas (CP-04, CP-22, CP-23, CA-04): 4 aprobadas, 0 fallidas, 0 omitidas. CA-04: APROBADO (8/8 pasos verificados en pantalla).
Estado	**Aprobada**
Evidencia	Captura 03; docs/evidencias/junit.xml

CT-09 — Temperatura de agua: histéresis hasta 105 °C

Identificador	CT-09
Requisitos	R04
Elementos	4
Técnica	Transición de estados (histéresis)
Precondiciones	Motor encendido.
Datos de prueba	115, 105.1, 105 y 104 °C
Pasos	1. Fijar 115 °C y esperar 500 ms. 2. Bajar a 105.1 °C y esperar 500 ms. 3. Bajar a 105 °C (o 104 °C) y esperar 500 ms.
Resultado esperado	Rojo en 115 °C; conserva rojo en 105.1 °C; normal en 105 °C o menos.
Resultado obtenido	3 ejecuciones automatizadas (CP-21, CP-28, CA-04): 3 aprobadas, 0 fallidas, 0 omitidas. CA-04: APROBADO (8/8 pasos verificados en pantalla).
Estado	**Aprobada**
Evidencia	docs/evidencias/aceptacion.txt (CA-04); docs/evidencias/junit.xml

CT-10 — Presión de turbo: contorno ámbar sobre 1.5 bar

Identificador	CT-10
Requisitos	R05
Elementos	5
Técnica	Valores límite
Precondiciones	Motor encendido.
Datos de prueba	1.0, 1.5 y 1.51 bar
Pasos	1. Fijar 1.0 bar y esperar 100 ms. 2. Fijar 1.5 bar y esperar 100 ms. 3. Fijar 1.51 bar y esperar 100 ms.
Resultado esperado	1.0 y 1.5 bar sin advertencia; 1.51 bar con contorno ámbar.
Resultado obtenido	4 ejecuciones automatizadas (CP-05, CP-25, CP-26, CA-05): 4 aprobadas, 0 fallidas, 0 omitidas. CA-05: APROBADO (6/6 pasos verificados en pantalla).
Estado	**Aprobada**
Evidencia	Captura 03; docs/evidencias/junit.xml

CT-11 — Presión de turbo: histéresis hasta 1.4 bar

Identificador	CT-11
Requisitos	R05
Elementos	5
Técnica	Transición de estados (histéresis)
Precondiciones	Motor encendido.
Datos de prueba	1.6, 1.41, 1.4 y 1.3 bar
Pasos	1. Fijar 1.6 bar y esperar 100 ms. 2. Bajar a 1.41 bar. 3. Bajar a 1.4 bar (o 1.3 bar).
Resultado esperado	Ámbar en 1.6 bar; conserva ámbar en 1.41 bar; sin advertencia en 1.4 bar o menos.
Resultado obtenido	3 ejecuciones automatizadas (CP-24, CP-29, CA-05): 3 aprobadas, 0 fallidas, 0 omitidas. CA-05: APROBADO (6/6 pasos verificados en pantalla).
Estado	**Aprobada**
Evidencia	docs/evidencias/aceptacion.txt (CA-05); docs/evidencias/junit.xml

CT-12 — Testigo 9: encendido tras 1 s bajo 1 bar

Identificador	CT-12
Requisitos	R02
Elementos	2, 9
Técnica	Valores límite (tiempo)
Precondiciones	Motor encendido.
Datos de prueba	0.99 bar sostenido 999 ms y 1000 ms; 0.5 bar
Pasos	1. Fijar 0.99 bar. 2. Observar el testigo 9 a los 999 ms de presión baja. 3. Observarlo a los 1000 ms. 4. Comparar con el medidor 2 en el mismo instante.
Resultado esperado	Testigo 9 apagado a los 999 ms y rojo fijo desde los 1000 ms; el medidor 2 ya estaba en ámbar desde el primer ciclo.
Resultado obtenido	4 ejecuciones automatizadas (CP-17, CP-18, CP-70, CA-09): 4 aprobadas, 0 fallidas, 0 omitidas. CA-09: APROBADO (12/12 pasos verificados en pantalla).
Estado	**Aprobada**
Evidencia	Captura 04; docs/evidencias/junit.xml

CT-13 — Testigo 9: histéresis de apagado

Identificador	CT-13
Requisitos	R02
Elementos	9
Técnica	Transición de estados (histéresis)
Precondiciones	Motor encendido.
Datos de prueba	1.0, 1.1 y 1.2 bar
Pasos	1. Con el testigo apagado, mantener 1.0 bar más de 1 s. 2. Encender el testigo (0.5 bar durante 1 s) y subir a 1.1 bar. 3. Subir a 1.2 bar y observar a los 900 y 1000 ms.
Resultado esperado	1.0 bar no lo enciende; en 1.1 bar sigue encendido; con 1.2 bar se apaga al cumplirse 1 s.
Resultado obtenido	3 ejecuciones automatizadas (CP-19, CP-20, CA-09): 3 aprobadas, 0 fallidas, 0 omitidas. CA-09: APROBADO (12/12 pasos verificados en pantalla).
Estado	**Aprobada**
Evidencia	docs/evidencias/aceptacion.txt (CA-09); docs/evidencias/junit.xml

CT-14 — Direccionales: alternancia cada 500 ms

Identificador	CT-14
Requisitos	R06
Elementos	6
Técnica	Transición de estados
Precondiciones	Motor encendido.
Datos de prueba	IZQ, DER y OFF
Pasos	1. Seleccionar IZQ. 2. Observar el testigo 6 en t = 0, 500 y 1000 ms. 3. Cambiar a DER. 4. Cambiar a OFF.
Resultado esperado	Verde alternando cada 500 ms con IZQ y con DER; apagado con OFF.
Resultado obtenido	6 ejecuciones automatizadas (CP-06, CP-07, CP-08, CP-30, CP-57, CA-06): 6 aprobadas, 0 fallidas, 0 omitidas. CA-06: APROBADO (7/7 pasos verificados en pantalla).
Estado	**Aprobada**
Evidencia	Captura 05; docs/evidencias/junit.xml

CT-15 — Luces altas: azul fijo

Identificador	CT-15
Requisitos	R07, R01
Elementos	7
Técnica	Funcional, transición de estados
Precondiciones	Motor apagado.
Datos de prueba	luces_altas = 1 / 0
Pasos	1. Activar luces altas con el motor apagado. 2. Encender el motor. 3. Observar el testigo 7 en t = 0, 250 y 500 ms. 4. Desactivar luces altas.
Resultado esperado	Apagado con el motor apagado (D2); azul fijo, sin parpadeo, con el motor encendido; apagado al desactivarlas.
Resultado obtenido	4 ejecuciones automatizadas (CP-64, CP-65, CP-72, CA-07): 4 aprobadas, 0 fallidas, 0 omitidas. CA-07: APROBADO (5/5 pasos verificados en pantalla).
Estado	**Aprobada**
Evidencia	Captura 05; docs/evidencias/junit.xml

CT-16 — Intermitentes de emergencia: alternancia cada 500 ms

Identificador	CT-16
Requisitos	R07
Elementos	8
Técnica	Transición de estados
Precondiciones	Motor encendido.
Datos de prueba	emergencia = 1 / 0
Pasos	1. Activar la emergencia. 2. Observar el testigo 8 en t = 0, 500 y 1000 ms. 3. Desactivar la emergencia.
Resultado esperado	Ámbar alternando cada 500 ms; apagado al desactivarla.
Resultado obtenido	3 ejecuciones automatizadas (CP-31, CP-58, CA-08): 3 aprobadas, 0 fallidas, 0 omitidas. CA-08: APROBADO (8/8 pasos verificados en pantalla).
Estado	**Aprobada**
Evidencia	Captura 06; docs/evidencias/junit.xml

CT-17 — Precedencia de la emergencia sobre el testigo de giro

Identificador	CT-17
Requisitos	R07, R06
Elementos	6, 8
Técnica	Transición de estados
Precondiciones	Motor encendido.
Datos de prueba	direccional = IZQ; emergencia = 1 / 0
Pasos	1. Seleccionar IZQ y comprobar que el testigo 6 se enciende. 2. Activar la emergencia. 3. Observar los testigos 6 y 8. 4. Desactivar la emergencia.
Resultado esperado	Con emergencia activa solo parpadea el 8 y el 6 no se ilumina (D7); al desactivarla, el 6 vuelve a mostrarse.
Resultado obtenido	4 ejecuciones automatizadas (CP-33, CP-34, CP-71, CA-08): 4 aprobadas, 0 fallidas, 0 omitidas. CA-08: APROBADO (8/8 pasos verificados en pantalla).
Estado	**Aprobada**
Evidencia	Captura 06; docs/evidencias/junit.xml

CT-18 — Falla de frenos: parpadeo cada 300 ms

Identificador	CT-18
Requisitos	R08
Elementos	10
Técnica	Transición de estados
Precondiciones	Motor encendido.
Datos de prueba	falla_frenos = 1 / 0
Pasos	1. Activar falla de frenos. 2. Observar el testigo 10 en t = 0, 300, 600 y 900 ms. 3. Desactivar falla de frenos.
Resultado esperado	Rojo alternando cada 300 ms; apagado con falla_frenos = 0.
Resultado obtenido	3 ejecuciones automatizadas (CP-32, CP-59, CA-10): 3 aprobadas, 0 fallidas, 0 omitidas. CA-10: APROBADO (6/6 pasos verificados en pantalla).
Estado	**Aprobada**
Evidencia	docs/evidencias/aceptacion.txt (CA-10); docs/evidencias/junit.xml

CT-19 — Falla de frenos con el motor apagado

Identificador	CT-19
Requisitos	R08, R01
Elementos	10
Técnica	Basada en requisitos
Precondiciones	Motor apagado.
Datos de prueba	falla_frenos = 1
Pasos	1. Activar falla de frenos sin encender el motor. 2. Observar el testigo 10 durante 900 ms. 3. Encender el motor y seguir observando.
Resultado esperado	El testigo 10 parpadea en rojo cada 300 ms aunque el motor esté apagado (D1) y no cambia al encender el motor.
Resultado obtenido	2 ejecuciones automatizadas (CP-66, CA-10): 2 aprobadas, 0 fallidas, 0 omitidas. CA-10: APROBADO (6/6 pasos verificados en pantalla).
Estado	**Aprobada**
Evidencia	Captura 07; docs/evidencias/junit.xml

CT-20 — Alertas críticas visualmente distintas

Identificador	CT-20
Requisitos	Especificación (interfaz y manejo de errores)
Elementos	9, 10
Técnica	Revisión de interfaz
Precondiciones	Aplicación iniciada.
Datos de prueba	Ninguno
Pasos	1. Observar la forma de los testigos 6–10. 2. Activar la falla de frenos.
Resultado esperado	Los testigos 9 y 10 se dibujan como rectángulos con borde rojo y la etiqueta CRITICA; los informativos 6–8 como círculos; el 10 se ilumina en rojo.
Resultado obtenido	2 ejecuciones automatizadas (CP-67, CP-76): 2 aprobadas, 0 fallidas, 0 omitidas.
Estado	**Aprobada**
Evidencia	Capturas 04 y 07; docs/evidencias/junit.xml

CT-21 — Entradas fuera de rango

Identificador	CT-21
Requisitos	R01, R02, R03, R04, R05
Elementos	1–5
Técnica	Partición de equivalencia (clases inválidas)
Precondiciones	Aplicación iniciada.
Datos de prueba	−1 y 361 km/h; 9001 RPM; 10.1 bar; 39.9 y 130.1 °C; 2.1 bar; 500 km/h
Pasos	1. Enviar cada valor al estado del clúster. 2. Comprobar la excepción y el estado posterior.
Resultado esperado	Cada valor se rechaza con ValueError y el último valor válido se conserva.
Resultado obtenido	8 ejecuciones automatizadas (CP-38, CP-39, CP-42, CP-43, CP-45, CP-46, CP-47, CP-52): 8 aprobadas, 0 fallidas, 0 omitidas.
Estado	**Aprobada**
Evidencia	docs/evidencias/pytest.txt; docs/evidencias/junit.xml

CT-22 — Tipos de dato inválidos

Identificador	CT-22
Requisitos	R01, R02, R03, R05, R06

Elementos	2, 3, 5, 6 y motor
Técnica	Partición de equivalencia (clases inválidas)
Precondiciones	Aplicación iniciada.
Datos de prueba	RPM 5000.0 y True; aceite "alta"; turbo True; direccional "ADELANTE" y 1; motor 1
Pasos	1. Enviar cada valor al estado del clúster. 2. Comprobar la excepción.
Resultado esperado	Tipos incorrectos se rechazan con TypeError y valores fuera del conjunto con ValueError.
Resultado obtenido	7 ejecuciones automatizadas (CP-40, CP-41, CP-44, CP-48, CP-49, CP-50, CP-51): 7 aprobadas, 0 fallidas, 0 omitidas.
Estado	**Aprobada**
Evidencia	docs/evidencias/pytest.txt; docs/evidencias/junit.xml

CT-23 — Error informado en la interfaz

Identificador	CT-23
Requisitos	Especificación (interfaz y manejo de errores)
Elementos	3
Técnica	Manejo de errores
Precondiciones	Aplicación iniciada.
Datos de prueba	RPM = 9001 enviado por programa
Pasos	1. Enviar 9001 RPM a la interfaz. 2. Observar el mensaje. 3. Enviar un valor válido (100 RPM).
Resultado esperado	Aparece "Entrada rechazada: …" sin cambiar el estado; con un valor válido el mensaje desaparece.
Resultado obtenido	1 ejecuciones automatizadas (CP-77): 1 aprobadas, 0 fallidas, 0 omitidas.
Estado	**Aprobada**
Evidencia	Captura 08; docs/evidencias/junit.xml

CT-24 — Manejo del archivo de escenarios

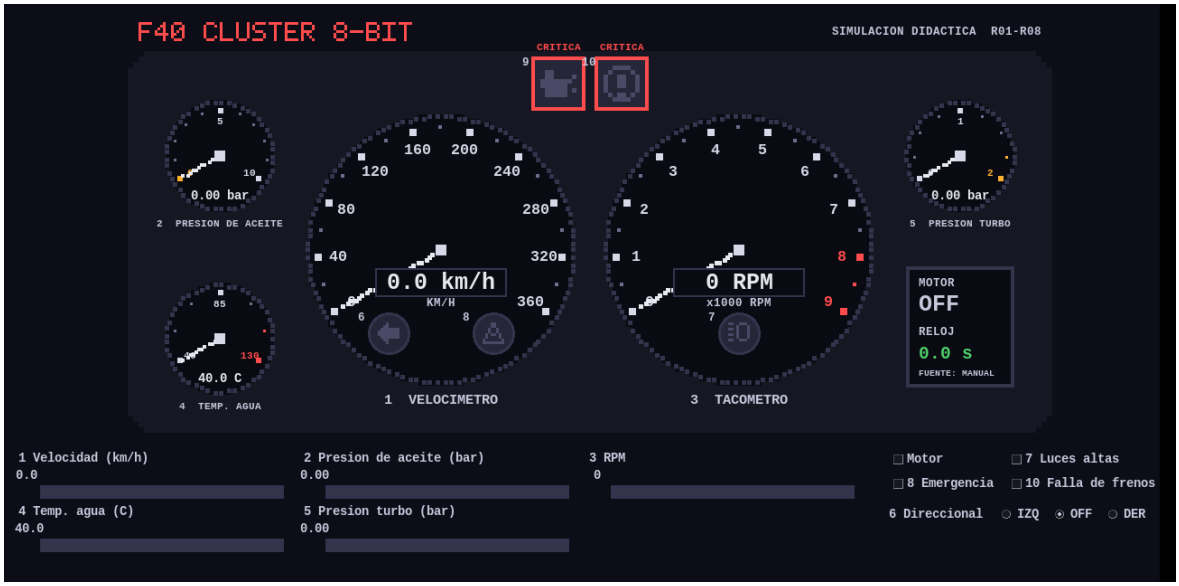
Identificador	CT-24
Requisitos	Especificación (interfaz y manejo de errores)
Elementos	—
Técnica	Manejo de errores y archivos
Precondiciones	Ejecutor de aceptación disponible.
Datos de prueba	Archivo inexistente, vacío, solo cabecera, sin columna esperado, control desconocido, valor no numérico y seis tipos de fila inválida
Pasos	1. Ejecutar el cargador con cada archivo. 2. Comprobar el mensaje de error y el código de salida.
Resultado esperado	Cada caso produce un error claro que indica la causa y, cuando aplica, la línea; el ejecutor devuelve 2.
Resultado obtenido	14 ejecuciones automatizadas (CP-79, CP-80, CP-81, CP-82, CP-83, CP-84, CP-87, CP-88): 14 aprobadas, 0 fallidas, 0 omitidas.
Estado	**Aprobada**
Evidencia	docs/evidencias/pytest.txt; docs/evidencias/junit.xml

CT-25 — Prueba de aceptación completa desde archivo

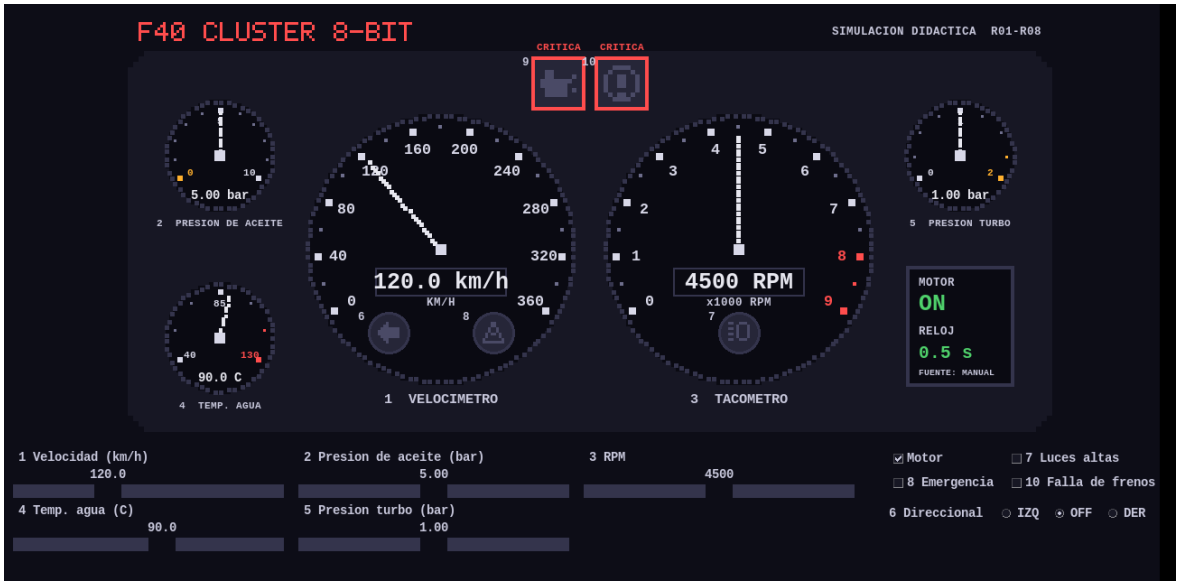
Identificador	CT-25
Requisitos	R01, R02, R03, R04, R05, R06, R07, R08
Elementos	1–10
Técnica	Aceptación dirigida por datos
Precondiciones	Archivo tests/datos/escenarios_aceptacion.csv.

Datos de prueba	10 escenarios (CA-01 a CA-10) que recorren los 10 controles
Pasos	1. Ejecutar: xvfb-run -a python -m aceptacion.escenarios tests/datos/escenarios_aceptacion.csv 2. Revisar el resultado de cada paso. 3. Comprobar que una expectativa incorrecta a propósito se detecta como FALLIDO.
Resultado esperado	10/10 escenarios aprobados y el ejecutor detecta una expectativa incorrecta.
Resultado obtenido	13 ejecuciones automatizadas (CP-85, CP-89, CP-90, CA-01, CA-02, CA-03, CA-04, CA-05, CA-06, CA-07, CA-08, CA-09, CA-10): 13 aprobadas, 0 fallidas, 0 omitidas. CA-01: APROBADO (6/6 pasos verificados en pantalla). CA-02: APROBADO (6/6 pasos verificados en pantalla). CA-03: APROBADO (8/8 pasos verificados en pantalla). CA-04: APROBADO (8/8 pasos verificados en pantalla). CA-05: APROBADO (6/6 pasos verificados en pantalla). CA-06: APROBADO (7/7 pasos verificados en pantalla). CA-07: APROBADO (5/5 pasos verificados en pantalla). CA-08: APROBADO (8/8 pasos verificados en pantalla). CA-09: APROBADO (12/12 pasos verificados en pantalla). CA-10: APROBADO (6/6 pasos verificados en pantalla).
Estado	**Aprobada**
Evidencia	docs/evidencias/aceptacion.txt; docs/evidencias/junit.xml

Evidencia visual



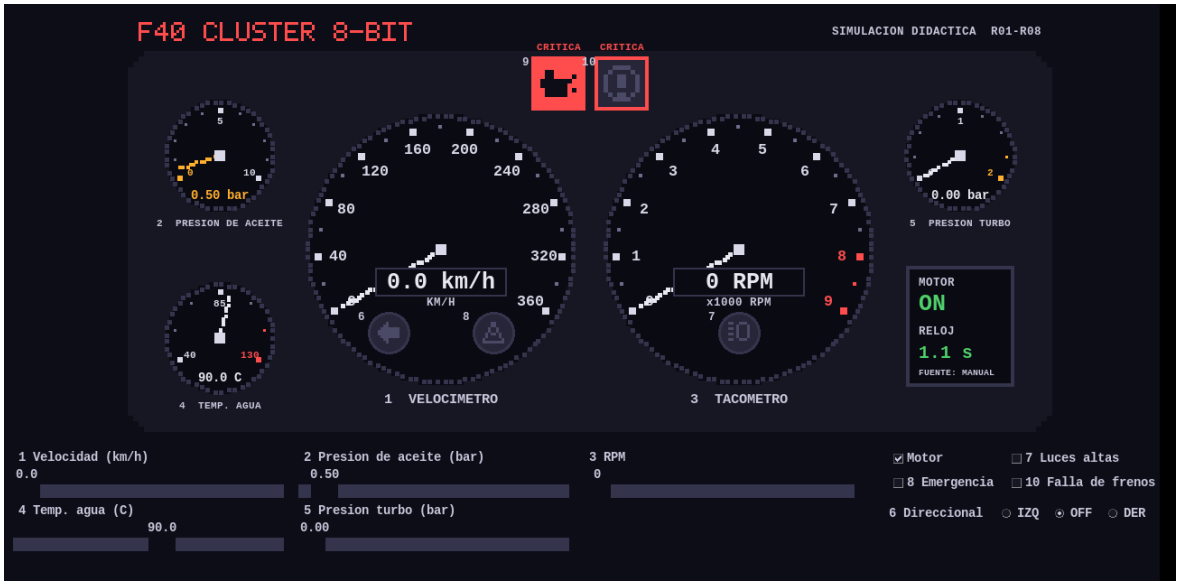
Captura 01. Estado inicial seguro: motor apagado, agujas en su minimo y testigos apagados (CA-01 a CA-10 (paso inicial), CP-68, CP-74 | Requisitos: R01).



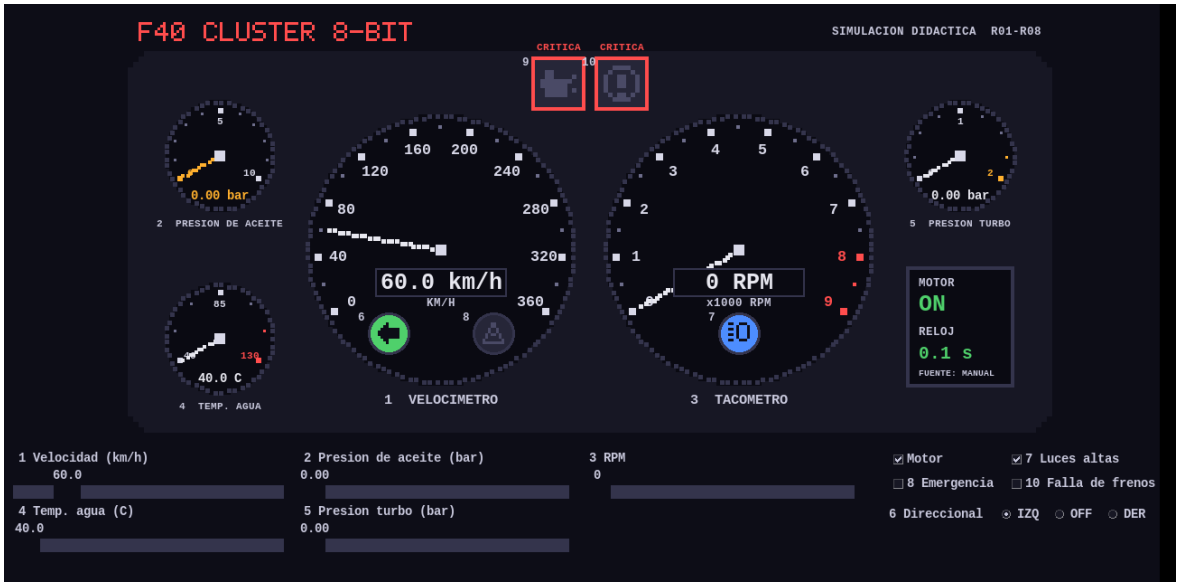
Captura 02. Motor encendido con lecturas dentro de rango: sin advertencias (CA-01, CA-03, CA-04, CA-05 | Requisitos: R01).



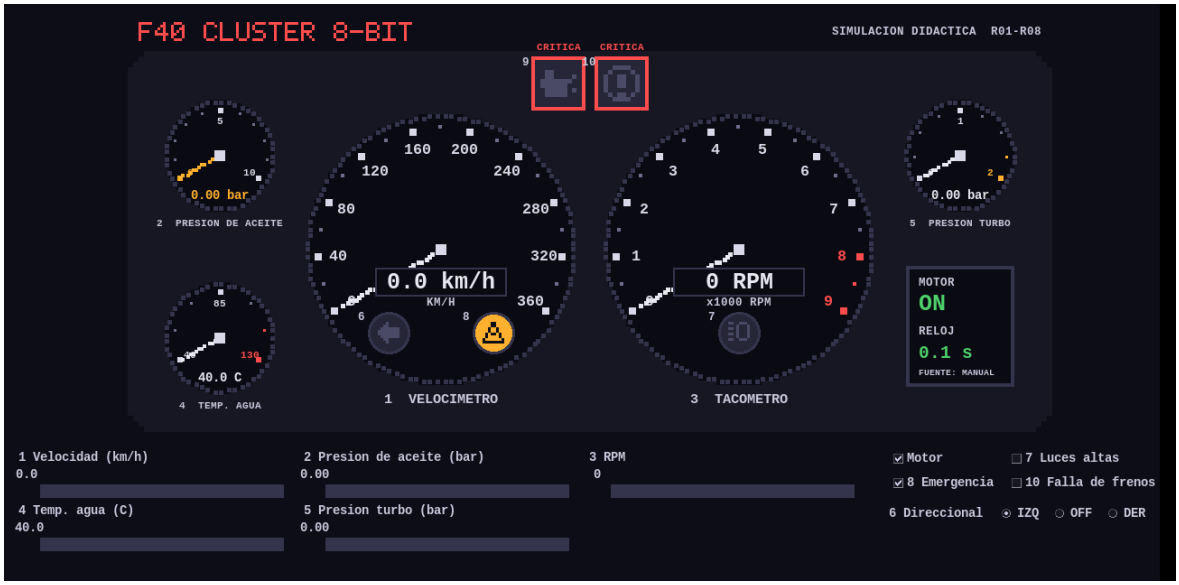
Captura 03. Umbrales cruzados: 2 y 5 en ambar, 3 y 4 en rojo (CA-02, CA-03, CA-04, CA-05, CP-69 | Requisitos: R03, R04, R05).



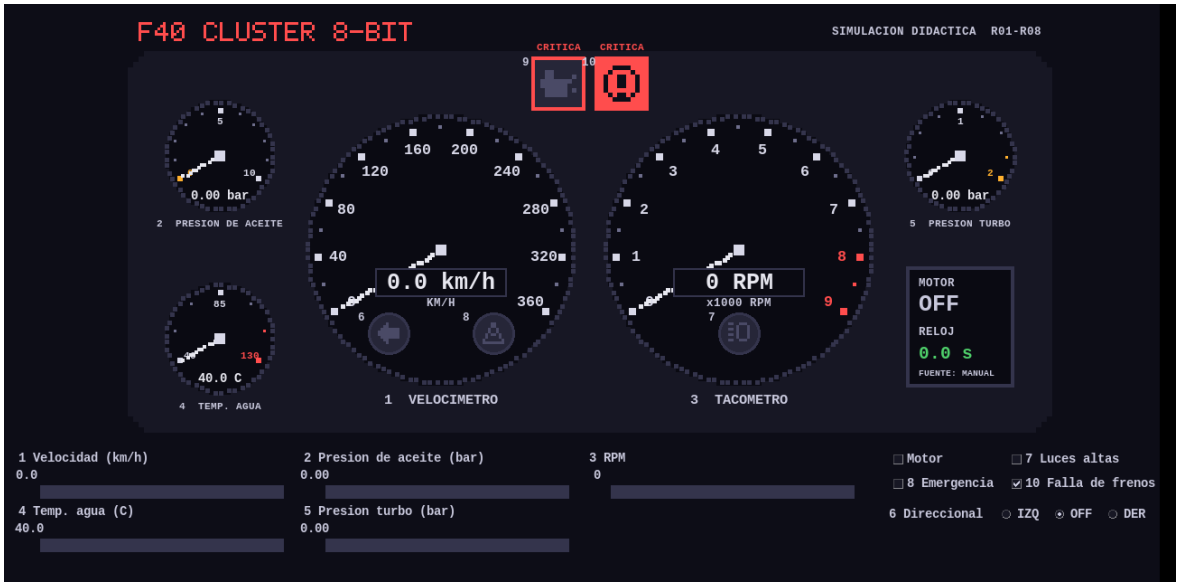
Captura 04. Presion de aceite bajo 1 bar por 1 s: testigo 9 rojo (critico) (CA-09, CP-70 | Requisitos: R02).



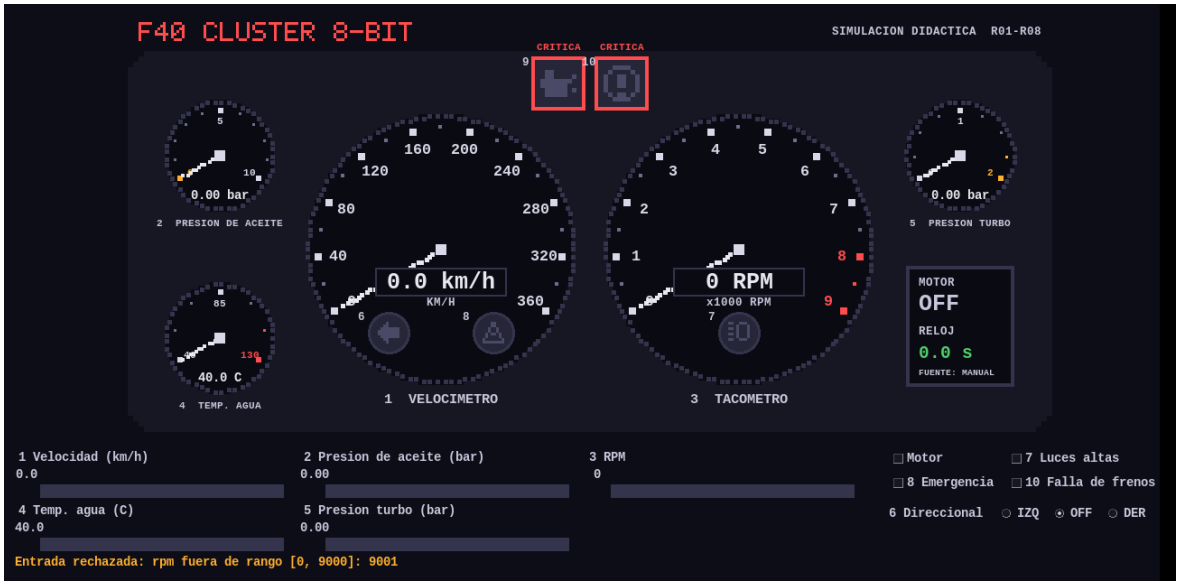
Captura 05. Direccional IZQ (6 verde) y luces altas (7 azul fijo) (CA-06, CA-07 | Requisitos: R06, R07).



Captura 06. Emergencia activa con direccional puesto: 8 ambar y 6 oculto (D7) (CA-08, CP-71 | Requisitos: R06, R07).



Captura 07. Falla de frenos con motor apagado: testigo 10 rojo (critico, D1) (CA-10, CP-76 | Requisitos: R08).



Captura 08. Valor fuera de rango (9001 RPM) enviado por programa: mensaje en pantalla (CP-77 | Requisitos: -).

10. Matriz de trazabilidad

Requisito	Casos del informe	Pruebas automatizadas (docs/matriz_trazabilidad.md)
R01	CT-01, CT-02, CT-03, CT-04, CT-05, CT-15, CT-19, CT-21, CT-22, CT-25	CA-01, CA-02, CA-03, CA-07, CA-09, CA-10, CP-35, CP-36, CP-37, CP-51, CP-53, CP-54, CP-65, CP-66, CP-68, CP-73, CP-74, CP-91, CP-92, CP-93, CP-94, CP-96, CP-115, CP-117
R02	CT-03, CT-12, CT-13, CT-21, CT-22, CT-25	CA-09, CP-03, CP-17, CP-18, CP-19, CP-20, CP-37, CP-43, CP-44, CP-70, CP-92, CP-93, CP-99, CP-117
R03	CT-06, CT-07, CT-21, CT-22, CT-25	CA-03, CP-02, CP-13, CP-14, CP-15, CP-16, CP-27, CP-40, CP-41, CP-42, CP-69, CP-75, CP-92, CP-93, CP-94, CP-115, CP-117
R04	CT-08, CT-09, CT-21, CT-25	CA-04, CP-04, CP-21, CP-22, CP-23, CP-28, CP-45, CP-46, CP-69, CP-92, CP-93, CP-94, CP-117
R05	CT-10, CT-11, CT-21, CT-22, CT-25	CA-05, CP-05, CP-24, CP-25, CP-26, CP-29, CP-47, CP-48, CP-69, CP-92, CP-93, CP-94, CP-117
R06	CT-14, CT-17, CT-22, CT-25	CA-06, CA-08, CP-06, CP-07, CP-08, CP-30, CP-49, CP-50, CP-57, CP-71
R07	CT-15, CT-16, CT-17, CT-25	CA-07, CA-08, CP-09, CP-31, CP-33, CP-34, CP-58, CP-64, CP-65, CP-71, CP-72, CP-117
R08	CT-18, CT-19, CT-25	CA-10, CP-09, CP-32, CP-59, CP-66, CP-72, CP-76, CP-99, CP-115, CP-117

Elementos y casos

Elemento	Casos del informe
1 Velocímetro	CT-01, CT-02, CT-03, CT-04, CT-21, CT-25
2 Presión de aceite	CT-01, CT-03, CT-05, CT-12, CT-21, CT-22, CT-25
3 Tacómetro	CT-01, CT-02, CT-03, CT-06, CT-07, CT-21, CT-22, CT-23, CT-25

Elemento	Casos del informe
4 Temperatura de agua	CT-01, CT-03, CT-08, CT-09, CT-21, CT-25
5 Presión de turbo	CT-01, CT-03, CT-10, CT-11, CT-21, CT-22, CT-25
6 Direccionales	CT-01, CT-14, CT-17, CT-25
7 Luces altas	CT-01, CT-15, CT-25
8 Intermitentes (emergencia)	CT-01, CT-16, CT-17, CT-25
9 Baja presión de aceite	CT-01, CT-03, CT-12, CT-13, CT-20, CT-25
10 Falla de frenos	CT-01, CT-18, CT-19, CT-20, CT-25

Requisitos sin casos: ninguno.

11. Registro de defectos

DEF-01 — El testigo de falla de frenos (10) parpadea con el motor apagado

ID	DEF-01
Estado	**Rechazado (no es defecto segun R08)**
Severidad asignada al reportarlo	Media
Requisitos relacionados	R01 (encendido/apagado), R08 (testigo de falla de frenos)
Componente	logic/estado.py, propiedad testigo_falla_frenos_encendido (testigo 10)
Detectado en	Etapa 4 (revision de codigo durante pruebas de caja blanca)
Caso de prueba	CP-66 (tests/test_requisitos.py)

Reporte original

- Crear EstadoCluster con el motor apagado (estado inicial).
- Llamar set_falla_frenos(True).
- Leer testigo_falla_frenos_encendido en t = 0 ms y cada 300 ms.

Se esperaba False con el motor apagado, interpretando R01 como un apagado general de todos los testigos. Se obtuvo True en t = 0 ms, alternando cada 300 ms.

Historial

- Etapa 6 (commit daad491): registrado como Abierto; CP-66 se marco xfail.
- Etapa 7 (commit 382a8d9): se agrego una condicion de motor al testigo 10 y el defecto se marco Cerrado.
- Etapa 8a.1: al revisar la especificacion completa se comprobo que:
- R08 dice "falla_frenos=1 -> testigo 10 rojo parpadeante cada 300 ms hasta falla_frenos=0", sin ninguna condicion de motor;
- R01 solo regula las lecturas continuas (velocidad, RPM, temperatura, presion de aceite y turbo);
- el elemento 10 esta clasificado como alerta CRITICA.

El comportamiento original cumplia R08. Se rechazo el defecto, se revirtio el cambio de la Etapa 7 y CP-66 ahora verifica que el testigo 10 parpadea con el motor apagado y se apaga con falla_frenos=0.

Leccion aprendida: la expectativa de un caso de prueba debe salir de la especificacion, no de la consistencia interna del codigo.

12. Resumen cuantitativo y conclusión de calidad

Indicador	Valor
Casos del informe (CT)	25: 25 aprobados, 0 fallidos, 0 bloqueados
Ejecuciones automatizadas (pytest)	145: 145 aprobadas, 0 fallidas, 0 omitidas
Escenarios de aceptación	10/10 aprobados (Resumen: 10/10 escenarios aprobados)
Capturas de evidencia	8
Defectos registrados	1 (0 abiertos)

Módulo	Líneas	Ramas	Cobertura	Ramas sin cubrir
logic/estado.py	191/191	64/64	100.0%	—
logic/reloj.py	18/18	4/4	100.0%	—
ui/presentacion.py	19/19	0/0	100.0%	—
ui/tablero.py	230/230	60/60	100.0%	—
main.py	56/56	7/8	98.4%	73->75
aceptacion/ escenarios.py	131/131	40/40	100.0%	—
sistema/config.py	69/69	16/16	100.0%	—
sistema/lector.py	210/214	64/70	96.5%	133->135, 135->salida, 144->146, 205->salida, 210->212, 220->224
sistema/bitacora.py	62/62	6/8	97.1%	47->49, 56->58
sistema/monitor.py	30/30	18/18	100.0%	—

Módulos con líneas o ramas sin cubrir: main.py, sistema/lector.py, sistema/bitacora.py.

Conclusión de calidad

Los 25 casos del informe están aprobados, las 145 ejecuciones automatizadas pasan, los 10 escenarios de aceptación coinciden con lo esperado en pantalla y no quedan defectos abiertos. Con los criterios de la sección 5, el software cumple R01–R08 y los comportamientos observables de los 10 elementos según la especificación y las decisiones de la sección 4. Las limitaciones principales son las de la sección 8: interpretación de la especificación, ejecución sobre pantalla virtual y tiempo simulado.

13. Referencias

[1] Especificación del proyecto Clúster F40, Sistemas Operativos, UAG (documento del curso).

[2] ISO/IEC/IEEE 29119-1:2022, Software and systems engineering — Software testing — Part 1: General concepts. https://www.iso.org/standard/81291.html (ficha verificada el 29/10/2026).

[3] ISO/IEC/IEEE 29119-2:2021, Software and systems engineering — Software testing — Part 2: Test processes. https://www.iso.org/standard/79428.html (ficha verificada el 29/10/2026).

[4] ISO/IEC/IEEE 29119-3:2021, Software and systems engineering — Software testing — Part 3: Test documentation. https://www.iso.org/standard/79429.html (ficha verificada el 29/10/2026).

[5] ISO/IEC/IEEE 29119-4:2021, Software and systems engineering — Software testing — Part 4: Test techniques. https://webstore.iec.ch/publication/71836 (ficha verificada el 29/10/2026).

[6] ISO 26262-6:2018, Road vehicles — Functional safety — Part 6: Product development at the software level. https://www.iso.org/standard/68388.html (ficha verificada el 29/10/2026). Solo como contexto; no se demuestra conformidad.

[7] pytest documentation. https://docs.pytest.org/en/stable/ (consultado el 29/10/2026).

[8] Coverage.py documentation. https://coverage.readthedocs.io/ (consultado el 29/10/2026).

[9] tkinter — Python interface to Tcl/Tk. https://docs.python.org/3/library/tkinter.html (consultado el 29/10/2026).

[10] Pillow documentation. https://pillow.readthedocs.io/ (consultado el 29/10/2026).

[11] python-docx documentation. https://python-docx.readthedocs.io/ (consultado el 29/10/2026).

[12] Xvfb(1), Ubuntu Manpage. https://manpages.ubuntu.com/manpages/jammy/man1/Xvfb.1.html (consultado el 29/10/2026).

[13] LibreOffice. https://www.libreoffice.org/ (consultado el 29/10/2026).

14. Anexo: uso de inteligencia artificial

Declaración de colaboración

Claude Code, asistente de inteligencia artificial de Anthropic, colaboró con el Equipo 3 en la realización de este proyecto: escribió el código del clúster y de su interfaz, diseñó y programó las pruebas y los escenarios de aceptación, preparó los scripts de evidencia, revisó las capturas y redactó este informe. Las decisiones sobre la especificación (D1–D7) las tomó el equipo, que además ejecutó cada etapa en su propio equipo y validó los resultados reales antes de continuar.

Detalle

En este proyecto se usaron asistentes de inteligencia artificial. Según la bitácora, las Etapas 0 a 3 se trabajaron con un asistente en otra sesión y las Etapas 4 a 8e con Claude Code (Anthropic). La IA generó el código, las pruebas, los valores esperados de los escenarios, los scripts de evidencia y este documento. El equipo ejecutó cada etapa en su máquina virtual, revisó la salida real antes de continuar y tomó las decisiones de interpretación D1–D7. Ningún resultado de este informe se escribió a mano: todos provienen de las ejecuciones guardadas en docs/evidencias/. La bitácora completa se transcribe a continuación.

Etapa 0 — Auditoría

- Fecha: 2026-10-29 00:13
- Qué se hizo: se auditó /home/ayocesaar/cluster-f40, confirmando que estaba vacía (sin trabajo previo). Se instalaron paquetes de sistema (python3-tk, python3-venv) con sudo del usuario. Se creó el entorno virtual .venv y se instalaron pytest, coverage, pillow y python-docx dentro de él. Se inicializó el repositorio git y se creó la estructura de carpetas base.
- Comandos ejecutados: ver docs/evidencias/pytest_inicial.txt y el historial de esta sesión.
- Resultado real: entorno listo, sin pruebas aún (no existían).
- Qué generó la IA: la estructura de carpetas, el contenido de este archivo y de docs/auditoria.md, y el .gitignore. El usuario (César) ejecutó todos los comandos en su VM y verificó la salida real en cada paso.

Etapa 1 y 2 — Entorno y logica central

- Entorno confirmado en la auditoria inicial: Ubuntu 22.04.5 LTS (jammy), Python 3.10.12, git 2.34.1, tkinter disponible tras instalar python3-tk.

- Se crearon logic/reloj.py (RelojReal, RelojSimulado) y logic/estado.py (EstadoCluster con R01-R08), sin dependencias de Tkinter.
- Decision de diseno documentada: al apagar el motor se apaga tambien el testigo 9 y el ambar del elemento 2, y se reinician sus temporizadores de histeresis, para evitar un falso encendido 1s despues de apagar el motor.
- Qué generó la IA: el contenido completo de ambos archivos. El usuario los guardó y verificó que compilan sin errores de sintaxis.

Etapa 3 — Suite de pruebas pytest

- Fecha: 2026-10-29
- Qué se hizo: 52 casos (CP-01 a CP-52) en tests/test_particiones.py, test_limites.py, test_transiciones.py y test_invalidas.py, con conftest.py que inyecta RelojSimulado.
- Resultado real: 52 passed; cobertura total 97% (estado.py 95%, reloj.py 73%). Commit e1b670a.
- Qué generó la IA: todo el código de pruebas. El usuario lo ejecutó en su VM y verificó la salida.
- Incidencias: la sesión SSH se cerró antes del commit; se reconectó y se comprobó con git status que no se perdió nada. Se corrigió el docstring de CP-27 (decía 7800, la prueba usa 7799).

Etapa 4 — Pruebas complementarias de caja blanca

- Qué se hizo: CP-53 a CP-63 para las líneas y ramas sin cubrir (motor_activo, color_aceite, testigos apagados, RelojReal, validaciones de RelojSimulado.avanzar). Se reforzaron CP-09, CP-31 y CP-32, que no verificaban todos los valores que prueban.
- Resultado real: 65 passed; cobertura 100% en logic/. Commit f7bcff0.
- Qué generó la IA: el análisis de las líneas sin cubrir (a partir del código que pegó el usuario) y las pruebas nuevas.

Etapa 5 — Matriz de trazabilidad

- Qué se hizo: tools/trazabilidad.py genera docs/matriz_trazabilidad.md desde los docstrings y el resultado real de pytest (junit.xml).
- Resultado real: 63 casos, 63 PASS. Commit 27d6997.
- Incidencias: el primer bloque falló porque la IA usó "!!" dentro de comillas dobles y bash lo interpretó como expansión de historial. Fue un error del comando, no del proyecto; se resolvió con set +H.

Etapa 6 — Trazabilidad completa y registro de defectos

- Qué se hizo: se etiquetaron con R01-R08 los casos sin requisito, se agregaron CP-64 a CP-66 y se registró DEF-01 (el testigo de frenos parpadeaba con el motor apagado) en docs/registro_defectos.md; CP-66 se marcó xfail para documentar el defecto sin romper la suite.
- Resultado real: 67 passed, 1 xfailed; ningún requisito sin casos. Commit daad491.
- Decisión: la IA propuso interpretar R01 ("encendido/apagado general") como apagado de todos los testigos, igual que los demás testigos del código. Debe confirmarse contra la guía original.

Etapa 7 — Corrección de DEF-01 e informe

- Qué se hizo: se agregó la verificación del motor en testigo_falla_frenos_encendido, se quitó el xfail de CP-66 (queda como prueba de regresión) y tools/informe.py genera docs/informe_pruebas.md con datos reales de junit, coverage y el registro de defectos.
- Resultado real: 68 passed, cobertura 100% en logic/, DEF-01 cerrado. Commit 382a8d9.
- Qué generó la IA (Etapas 4-7): Claude Code generó los comandos, pruebas, scripts y documentos. El usuario ejecutó cada bloque en su VM y pegó la salida real, que se revisó antes de continuar.

Etapa 8a.1 — Revisión contra la especificación y reversión de DEF-01

- Fecha: 2026-10-29

- Qué se hizo: el usuario proporcionó la especificación completa como texto. La IA comparó numeración, rangos, ciclos y reglas R01-R08 contra logic/estado.py; la numeración y los umbrales coinciden. Se identificaron seis puntos de decisión (D1-D6) y el usuario aceptó las recomendaciones.
- D1: DEF-01 se rechazó. La IA lo había registrado en la Etapa 6 interpretando R01 como apagado general; la especificación (R08) no condiciona el testigo 10 al motor. Fue un error de interpretación de la IA. Se revirtió el cambio de la Etapa 7 y CP-66 se reescribió con la expectativa correcta.
- D2: los testigos 6, 7 y 8 siguen dependiendo del motor, documentado como el estado definido de R01.
- D3: un solo interruptor motor representa Encendido y Motor activo.
- D4 y D5 se aplicarán en las Etapas 8c y 8e. D6: la portada usará la fecha de entrega de la especificación (18/10/2026).
- Qué generó la IA: el análisis comparativo, los cambios de código, prueba y documentación. El usuario tomó las decisiones y ejecutó el bloque en su VM.

Etapa 8b — Interfaz Tkinter y main.py

- Fecha: 2026-10-29
- Qué se hizo: ui/presentacion.py (traduce el estado a lo que se dibuja de los 10 elementos, sin Tkinter), ui/tablero.py (5 deslizadores, 4 interruptores, selector de direccional; agujas 1-5, testigos 6-10, críticos 9 y 10 como rectángulos con borde rojo) y main.py. Pruebas CP-67 a CP-78 (presentación e integración con Tkinter real bajo Xvfb).
- Decisión D7 (propuesta por la IA y aprobada por el usuario): la precedencia visual de R07 se implementa ocultando el testigo 6 mientras la emergencia está activa.
- Incidencias: el primer intento de pegar el bloque completo en la terminal SSH se corrompió por su tamaño (la terminal quedó esperando el cierre de un heredoc). Se cambió el método: la IA escribió los archivos en Windows, el usuario los copió a la VM con scp y ejecutó un script.
- Qué generó la IA: todo el código de la interfaz, sus pruebas y el script de la etapa. El usuario instaló Xvfb, copió los archivos y ejecutó el script en su VM.

Etapa 8c — Pruebas de aceptación dirigidas por datos

- Fecha: 2026-10-29
- Qué se hizo: aceptacion/escenarios.py carga escenarios desde tests/datos/escenarios_aceptacion.csv, los ejecuta sobre la interfaz Tkinter real (reloj simulado, ventana nueva en estado seguro por escenario) y compara lo dibujado en pantalla con lo esperado. Diez escenarios CA-01 a CA-10, uno por elemento, que recorren los 10 controles y cruzan cada umbral en ambos sentidos. Pruebas de manejo del archivo (CP-79 a CP-85, CP-87 a CP-90: inexistente, vacío, columnas faltantes, filas inválidas) y CP-86 para el hueco de cobertura de detener(). Decisión D4 aplicada.
- Evidencia: docs/evidencias/aceptacion.txt con la salida real paso a paso.
- Qué generó la IA: el cargador y ejecutor, el CSV de escenarios con sus valores esperados (calculados a partir de las reglas R01-R08 y de los ciclos), y las pruebas. El usuario copió los archivos a la VM y ejecutó el script; los resultados son los de esa ejecución.

Etapa 8d — Capturas de pantalla como evidencia

- Fecha: 2026-10-29
- Qué se hizo: tools/capturas.py abre la interfaz real sobre Xvfb (pantalla virtual de 1280x1024, 24 bits), la lleva a 8 estados representativos con el reloj simulado y fotografía la ventana con Pillow. Cada imagen se valida (no vacía) y se registra en docs/evidencias/capturas.txt junto con los casos y requisitos que ilustra y el estado de los 10 elementos en ese instante.

- Qué generó la IA: el script y la selección de estados. Las imágenes son las que produjo la ejecución en la VM del usuario, sin edición posterior.

Etapa 8e — Ajuste visual e informe Word/PDF

- Fecha: 2026-10-29
- Ajuste visual: al revisar las capturas de la 8d se vio que la aguja tapaba la cifra en el mínimo y que los interruptores tenían un recuadro de foco que parecía tachar el texto. Se corrigió en ui/tablero.py, se repitió la regresión completa y se regeneraron las capturas.
- Informe: informe/generar_informe.py produce Informe_Pruebas_Cluster_F40.docx y .pdf con la estructura de 14 secciones de la especificación y 25 casos consolidados (CT-01 a CT-25). Los resultados, conteos, versiones y porcentajes se leen de docs/evidencias/; el estado de cada CT se calcula de junit.xml y aceptacion.txt. Las referencias en línea se verificaron el 29/10/2026; las de iso.org quedaron como pendientes de verificar porque el sitio rechazó la consulta automática.
- Qué generó la IA: el ajuste visual, el generador del informe y su texto. Antes de entregarlo se probó el generador en Windows con evidencias de prueba (no reales) para revisar el formato. El usuario ejecutó la etapa en su VM; el documento final usa solo las evidencias reales de esa ejecución.

Etapa 8e.1 — Corrección visual detectada en el informe

- Fecha: 2026-10-29
- Qué se hizo: al revisar el PDF generado se vio que en las capturas el texto "7 Luces altas" salía casi negro: en Xvfb el puntero queda sobre ese interruptor y Tkinter usaba el color de texto activo por defecto. Se fijó activeforeground en interruptores y selector, se repitió la regresión completa, se regeneraron capturas y evidencias, y se volvió a generar el informe Word/PDF.
- Qué generó la IA: el diagnóstico (a partir del PDF) y el parche. El usuario ejecutó la etapa en su VM.

Etapa 8e.2 — Normas ISO y declaración de colaboración

- Fecha: 2026-10-29
- Qué se hizo: se verificaron las fichas oficiales de ISO/IEC/IEEE 29119-1:2022, 29119-2:2021, 29119-3:2021, 29119-4:2021 e ISO 26262-6:2018 mediante búsqueda web (iso.org rechazaba la descarga directa), se quitaron las marcas de "pendiente de verificar", se agregó al informe una tabla de marco normativo (cómo se aplicó cada norma, sin declarar conformidad) y una declaración breve de la colaboración de Claude Code. Se regeneró el informe Word/PDF con las mismas evidencias.
- Qué generó la IA: la verificación de referencias y los cambios al generador. El usuario ejecutó la etapa en su VM.

Etapa 9 — Interfaz 8-bit y apertura en Windows, Linux y macOS

- Fecha: 2026-11-09
- Qué se hizo: se rediseñó ui/tablero.py con estética 8-bit inspirada en la distribución del tablero del F40 (velocímetro y tacómetro grandes al centro, medidores chicos a los lados, testigos críticos 9 y 10 arriba entre las esferas, testigos 6, 7 y 8 dentro de las esferas), con agujas, esferas y escalas hechas de bloques de pixeles. Se cambió la paleta en ui/presentacion.py. Se agregaron abrir_cluster.bat (Windows), abrir_cluster.sh (Linux/macOS) y README.md con instrucciones.
- Pruebas: no se modificó ninguna prueba. La interfaz conserva lo que las pruebas observan (color de aguja y de testigo, texto de cada lectura, círculos para 6-8 y rectángulos para 9-10, controles y mensajes). Antes de llevarlo a la VM se corrió la suite completa en Windows: 107 aprobadas y sin fallos atribuibles al código; los fallos intermitentes observados se debieron al Python de Microsoft Store, que a veces no puede leer su propio init.tcl. La ejecución oficial es la de la VM.
- Qué generó la IA: el rediseño, los lanzadores, el README y la actualización del informe. El usuario pidió el cambio, encendió la VM y autorizó la llave SSH con la que la IA ejecutó la etapa.

Etapa 9.1 — Ajuste visual de la cifra digital

- Fecha: 2026-11-09
- Qué se hizo: al revisar las capturas de la Etapa 9 se vio que, con la aguja en cero, esta cruzaba la cifra digital del velocímetro y del tacómetro. Se agregó una ventana tipo LCD detrás de la cifra, por encima de la aguja. No se modificó ninguna prueba; se repitió la regresión completa.
- Qué generó la IA: el diagnóstico (a partir de la captura) y el ajuste. La IA ejecutó la etapa en la VM por SSH con la llave autorizada por el usuario.

Etapa 10 — Material de estudio y comparación con la guía oficial

- Fecha: 2026-11-09
- Qué se hizo: el usuario compartió la guía oficial del proyecto final. Se agregó la carpeta estudio/ con la guía en PDF, cómo abrir el proyecto en la VM, una guía de estudio para la demostración y una comparación punto por punto con la guía. La comparación muestra pendientes: datos desde archivos externos, configuración, bitácoras eventos.log/errores.log, comunicación y sincronización, permisos, pruebas P08, P11 y P12 e informe técnico con las secciones 4.1–4.7.
- Qué generó la IA: los documentos de estudio. Los pendientes quedan a decisión del equipo.

Etapa 11 — Proyecto completo según la guía oficial

- Fecha: 2026-11-10
- Qué se hizo: se construyó lo que pedía la guía del proyecto final y faltaba: - entrada por archivo externo (datos/vehiculo.conf, clave=valor) con un hilo lector que valida cada clave y conserva el último valor válido; - configuración externa (config/cluster.ini) con rutas relativas, intervalos, rangos y rotación; - bitácoras logs/eventos.log (INFO, WARN) y logs/errores.log (ERROR) con fecha, nivel, componente, evento y resultado, rotación por tamaño y permisos 640/750; - comunicación productor-consumidor (archivo con escritura atómica → hilo lector → queue.Queue(maxsize=1) → interfaz) y monitor de eventos de seguridad; - iniciar.sh, tools/escribir_dato.py, tools/simulador.py (proceso productor), tools/ejecucion_prolongada.py (P12 con métricas) y tools/demo_bitacoras.py; - pruebas P01–P12 y de sistema (CP-91 a CP-119) sin modificar ninguna prueba anterior; - diagramas, informe técnico con las 10 secciones de la guía y material de estudio actualizado.
- Hallazgo: en Windows, abrir el archivo justo cuando otro proceso lo reemplaza da un PermissionError momentáneo; la prueba P08 lo detectó y el lector ahora reintenta antes de registrarlo.
- Qué generó la IA: todo el código, las pruebas y los documentos de esta etapa. Se probó primero en Windows y la ejecución oficial fue en la VM por SSH; luego se subió a GitHub.